

GAME NETWORKING – LEC/LAB

LESSON 3 - PRELIM MINI PROJECT

MINI PROJECT:

Learning Objectives:

By completing this Prelim Exam Mini-Project, student pairs will demonstrate the ability to:

- **Network Initialization & Topology:** Establish network host and client connections using Unity (Netcode for GameObjects, Mirror, or Photon) or Godot (High-Level Multiplayer / ENet) architecture.
- **Data Serialization & Remote Procedure Calls (RPCs):** Serialize text payloads and send RPCs/Commands across network sockets to synchronize state across connected clients.
- **Chat System Architecture:** Implement essential messaging patterns including broadcast, channel filtering, direct whispering, and system notifications.
- **UI & Network State Synchronization:** Dynamically bind networking events to frontend UI components and handle network state changes (connection, disconnection, latency spikes).

Business Scenario:

Client: *NeonThread Interactive (Indie Game Studio)*

Project Code: *Project CyberLobby Subsystem*

NeonThread Interactive is in pre-production for an upcoming co-op cyberpunk immersive sim. Before building high-bandwidth 3D character synchronization, the lead networking architect requires a lightweight, rock-solid in-game terminal interface. This terminal will serve as the core communication layer between players during co-op hacking raids, allowing them to send real-time text transmissions, inspect network latency, and execute local terminal commands.

Your development team (Group of 2) has been contracted to build a functional prototype of this client-server terminal subsystem. The client requires a zero-desync messaging architecture capable of handling remote procedure calls (RPCs), local command execution, and gracefully handling connection states.

Project Description:

The **CyberLobby: Cyberpunk Terminal Chat** mini-project challenges student pairs to architect a host-authoritative, multi-client text terminal application using **Unity (Netcode for GameObjects / Mirror)** or **Godot (ENet / High-Level Multiplayer)**.

The objective is to establish a low-level understanding of packet transmission, client-server authority, remote procedure calls, and local input parsing within a stylized, dark-mode terminal user interface.

Core System Requirements:

- **Network Architecture & State Initialization:**
 - Support both **Host** (Server + Client) and **Client** runtime modes.

Date: August 10, 2026

- Dynamically assign a temporary, unique handle (e.g., User_482) to each connecting client.
- Provide visual connection and disconnection feedback directly in the terminal stream.
- **Server-Authoritative Messaging (RPC Pipeline):**
 - When a player submits a message, the client routes a ServerRpc / Command containing the message payload and sender handle to the server.
 - The server validates the payload and broadcasts a ClientRpc / TargetRpc to all connected clients to render the message in real-time.
- **Client-Side Command Engine:**
 - Implement local command processing (prefixed with /) that executes locally without polluting the network bandwidth:
 - /nick <new_handle> — Updates the local player's handle for subsequent broadcast messages.
 - /clear — Flushes the local terminal output display buffer.
 - /ping — Calculates and prints the current local frame time and network latency status.
- **Cyberpunk Terminal UI/UX:**
 - Styled with a dark background (#0D0D12) and neon green monospaced text (#00FF66).
 - Features an auto-scrolling log view (TextMeshPro / RichText) paired with an automatic input focus system for seamless command entry.

List of Deliverables (in MS Teams):

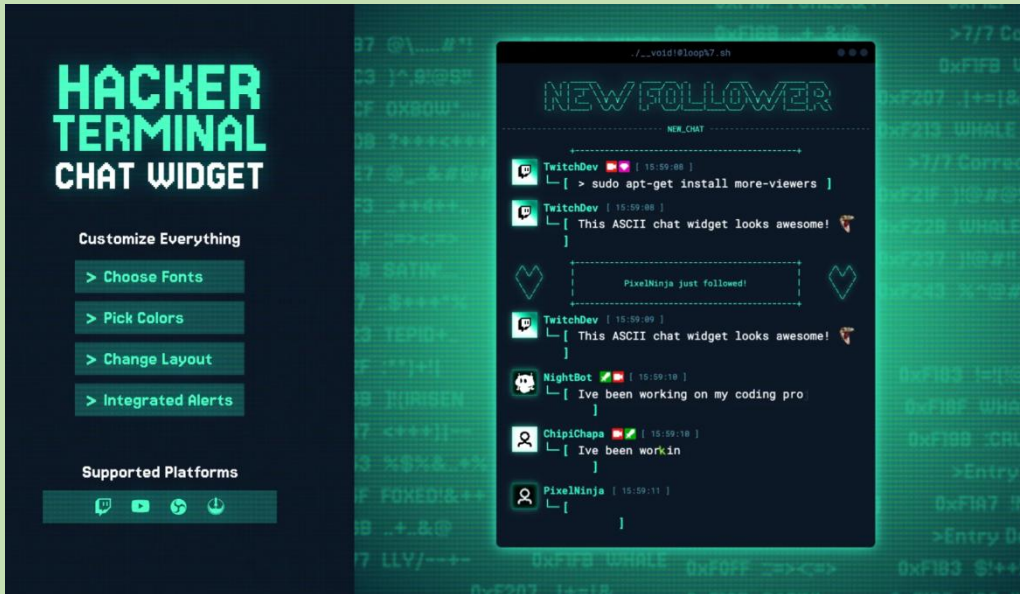
Submit the following in your designated MS Teams assignment bin on or before **August 18, 2026**:

1. **Repository Link:** URL to a clean public GitHub or GitLab repository with a configured .gitignore (no build files or Library folders).
2. **Build Deliverables:** Compressed folder (.zip) containing standalone executable builds (Host/Server build and Client build) for testing.
3. **Demonstration Video:** A 3 to 5-minute video showing two instances running side-by-side, demonstrating connection setup, core chat mechanics, and edge-case handling (e.g., disconnection).
4. **Project Documentation (PDF):**
 - High-level network architecture diagram (Client-Server or Peer-to-Peer topology).
 - RPC/Message data flow diagram showing how a message travels from Client A → Server → Client B.
 - Member contribution matrix signed by both partners.

Expected Outcome:

A fully functional 2-player or multi-client networked application developed in Unity or Godot. The application must feature a reliable connection setup, stable network state management, and a feature-complete multiplayer chat system that operates smoothly across multiple application instances without desynchronization or crashes.

Sample UI:



List of Mini-Project Titles & Descriptions:

1. CyberLobby: Cyberpunk Terminal Chat

A hacker-themed terminal lobby where players connect via IP and port. Includes terminal-style chat commands (/nick, /clear, /ping) and color-coded system response logs broadcasted via RPCs.

2. Tavern Keeper: Fantasy Guild Chat System

A fantasy-themed UI allowing players to switch between global channels (General, Trade, and Party). The network system filters chat RPC payloads so messages only render for players currently tuned to that specific channel.

3. Space Station Radio: Channel Frequency Chat

A sci-fi radio console where players enter numeric radio frequencies (e.g., 101.5 MHz). Only clients set to the same frequency receive broadcasted audio-style text messages and status alerts.

4. Retro RPG Combat Log & Party Chat

A dual-window chat architecture that separates raw system combat alerts (e.g., "Player A took 10 damage") from player-typed party messages, enforcing structured packet types over the network.

5. Pixel Plaza: Spatial Proximity Chat

A 2D top-down lobby where text bubbles only appear above a player's avatar if the receiving client is within a specific distance radius, using distance checks before broadcasting packets.

Date: August 10, 2026

6. Secret Agent: Encrypted Whisper Network

A stealth-themed chat featuring direct private messaging (/w username message). Messages are routed directly to the target client ID via targeted RPCs while ignoring untargeted clients.

7. Tournament Arena: Lobby & Match Chat System

A two-tier chat system where players join a master lobby channel, but upon entering a 1v1 match room, their chat instances switch exclusively to an isolated match-room network group.

8. Dungeon Crawler: Faction-Based Broadcast System

An asymmetric chat application where players select an "Order" or "Chaos" faction. Faction members share a private faction broadcast channel alongside a shared, neutral area chat.

9. Trading Post: Auction & Bargain Chat Box

A commerce-focused chat UI supporting structured item-link tags (e.g., [Item: Sword]) and a /bid <amount> custom command parsing system validated by the host server.

10. Vaporwave Lounge: Rich Text & Emote Chat

A stylized lounge chat featuring rich text (BBCode/HTML formatting) and a custom emote system that parses string tags (e.g., :smile:) into networked animated sprite popups.

11. Escape Room: Co-op Clue Transmission System

A cooperative puzzle chat where players send time-stamped log messages, structured puzzle hints, and interactive "pin" alerts to help each other solve room puzzles remotely.

12. Ship Captain: Fleet Semaphore & Broadcast UI

A naval-themed chat system that prioritizes messages into standard chat, high-priority fleet broadcasts, and emergency distress signals (/mayday) that flash red across all client screens.

13. Cyber Threat Matrix: Moderator & Filter Chat

A chat system featuring a host-authoritative bad-word filter, auto-censorship RPC pipeline, and host admin commands (/kick, /mute) to manage active client sessions.

14. Arcade Central: Spectator & Player Chat Room

An arcade lobby separating active players from spectators. Spectator messages are restricted to a dedicated spectator channel to prevent "ghosting" or giving away player positions.

15. Haunted Mansion: Ghost Realm Channel Chat

An asymmetric chat where dead/eliminated players transition to a "Ghost Realm" channel. Ghost messages are visible to other ghosts, but hidden from living players until the round ends.

16. Radio Tower: Network PING & Latency Indicator Chat

A technical chat box UI that calculates and displays real-time Round Trip Time (RTT/Ping) for every connected user next to their chat handle, flagging high-latency users.

17. Strategy Command: Tactical Map Ping & Chat

Date: August 10, 2026

A real-time tactical chat where players can send standard text or click a minimap to send synced coordinate map-ping notifications alongside location-based text alerts.

18. Monster Hunter: Quick-Wheel Radio Chat

An action-oriented chat combining a pre-set canned response wheel (RPC-triggered quick phrases like "Help!", "Heal Me!") with a standard manual text input field.

19. Broadcast News: Mayor's Ticker & Public Chat

A city-builder themed chat box containing a scrolling, server-authoritative headline ticker banner for major events, running concurrently with a public player chat stream.

20. Wild West Duel: Saloon Roleplay Chat

A roleplay chat system featuring local spatial proximity text, an action/emote command system (/me tipping hat), and saloon host moderation features.

Common Grading Rubric:

Criteria	Outstanding (90-100%)	Satisfactory (75-89%)	Needs Improvement (50-74%)	Unsatisfactory (<50%)	Weight
Network Architecture & Connection Handling	Host/Client initialization is robust. Disconnections and reconnections are handled cleanly without crashing.	Network establishes reliably. Minor unhandled edge cases during disconnects or invalid IP entry.	Connection works but frequently desynchronizes or throws console errors on startup.	Network fails to establish connection or crashes the build.	25%
Chat System Implementation	All core and specialized chat features (channels, commands, whispers, system alerts) work perfectly across clients.	Core chat works well; minor bugs present in specialized features or command parsing.	Basic message broadcasting works, but channel filtering or commands fail.	Chat system fails to send or receive messages across instances.	25%
Code Structure & Serialization	Excellent use of RPCs/Commands, efficient packet	Good use of networking tools; slight	Network code is disorganized, overuses heavy	Code is copied/broken; violates basic	20%

Date: August 10, 2026

Criteria	Outstanding (90-100%)	Satisfactory (75-89%)	Needs Improvement (50-74%)	Unsatisfactory (<50%)	Weight
	payloads, clear code layout, and proper network variable synchronization.	inefficiencies in serialization or message broadcasting logic.	string parameters, or lacks proper authority checks.	networking practices.	
UI/UX & Polish	UI is well-themed, responsive, auto-scrolls correctly, and provides clear visual feedback for connection states.	UI is functional and clean, with minor issues in auto-scrolling or text field focus management.	Basic functional UI with no styling, unreadable text, or alignment issues.	UI is non-functional, broken, or overlaps unreadably.	15%
Documentation & Deliverables	Includes clean repo, working builds, video demo, complete architecture diagram, and data flow charts.	All items were submitted, but the documentation lacks depth, and the architecture diagrams are simplified.	Missing minor deliverables (e.g., build issue or incomplete PDF diagram).	Major deliverables missing, non-functional builds, or late submission.	15%

Due date: August 18, 2026